

# **ABAP 7.4 CODING & TIPS**

# INTRODUCTION TO ABAP 7.4

- With the dawn of HANA the need raised to further develop ABAP 7.40 to make ABAP Codes fit for HANA.
- ABAP 7.4 provides many new features as below:
  - ✓ Inline Declarations
  - ✓ Table Expressions
  - ✓ Conversion Operator CONV
  - ✓ Value Operator
  - ✓ FOR operator
  - ✓ Conditional operators COND and SWITCH.

# USE OF READ STATEMENT:

## 1. Reading table (Single key)

### \* Old Syntax

```
READ TABLE IT_SCARR INTO LS_SCARR WITH KEY CARRID = 'AA'.
```

### \* New Syntax

```
DATA(LS_SCARR) = IT_SCARR[ CARRID = 'AA' ].
```

## 2. Reading table (Multiple key)

### \* Old Syntax

```
READ TABLE IT_SCARR INTO LS_SCARR WITH KEY CARRID = 'AA' CONNID = '0017'.
```

### \* New Syntax

```
DATA(LS_SCARR) = IT_SCARR[ CARRID = 'AA' CONNID = '0017' ].
```

# USE OF READ STATEMENT: (CONT...)

## 4. Use of Read statement similar to Transporting no fields with KEY.

### \* Old Syntax

```
READ TABLE LT_RETURN_CODES TRANSPORTING NO FIELDS WITH KEY FAILED =  
ABAP_TRUE.
```

### \* New Syntax

```
IF LINE EXISTS( LT_RETURN_CODES[ FAILED = ABAP_TRUE ] ).
```

“Success Logic

```
ENDIF.
```

## 3. Index Read:

### \* Old Syntax

```
READ TABLE IT_SCARR INTO LS_SCARR INDEX I.
```

### New Syntax:

```
DATA(LS_SCARR) = IT_SCARR[I].
```

**Note:** The predicate function `LINE_EXISTS` cannot be used to determine the row number in a table index of a search key, since table expressions do not fill the system field `SY-TABIX`.

# NEW SELECT STATEMENT:

- **Old Syntax:**

```
DATA: LT_COCKPIT TYPE TABLE OF ZEWM_HOSP_COCKP.  
  
SELECT * FROM ZEWM_HOSP_COCKP  
      INTO TABLE LT_COCKPIT  
      WHERE LGNUM EQ IV_LGNUM  
      AND CUSTOMS_HOLD EQ ABAP_TRUE.  
IF SY-SUBRC IS INITIAL.  
  SORT LT_COCKPIT BY PRCES PROCS.  
ENDIF.
```

- **\* New Syntax:**

```
SELECT * FROM ZEWM_HOSP_COCKP  
      INTO TABLE @DATA(LT_COCKPIT)  
      WHERE LGNUM EQ @IV_LGNUM  
      AND CUSTOMS_HOLD EQ @ABAP_TRUE.  
IF SY-SUBRC IS INITIAL.  
  SORT LT_COCKPIT BY PRCES PROCS.  
ENDIF.
```

**Note:** Here no need to define Internal Table LT\_COCKPIT using Data statement.

*'@' is mandatory to pass in where clause.*



# APPEND VALUE INTO ANOTHER TABLE USING FOR

## 2. Range from Internal table

### **\*Old Syntax:**

```
DATA: LT_RANGE TYPE RANGE OF RSDSSELOPT-LOW,  
      LS_RANGE LIKE LINE OF LT_RANGE.  
LOOP AT IT_TABLE ASSIGNING <FIELD_SYMBOL>.  
  LS_RANGE-SIGN      = WMEGC_SIGN_INCLUSIVE.  
  LS_RANGE-OPTION    = WMEGC_OPTION_EQ.  
  LS_RANGE-LOW       = <FIELD_SYMBOL>-FIELDNAME.  
  APPEND LS_RANGE TO LT_RANGE.  
ENDLOOP.
```

### **\*New Syntax:**

```
DATA(LT_RANGE) = VALUE RSELECTION( FOR <FIELD_SYMBOL> IN IT_TABLE  
  ( SIGN      = WMEGC_SIGN_INCLUSIVE  
    OPTION    = WMEGC_OPTION_EQ  
    LOW       = <FIELD_SYMBOL>-FIELDNAME ) ).
```

# FOR WITH WHERE CONDITION:

## 3. Range from Internal table using where clause

### **\*Old Syntax:**

```
DATA: LT_RANGE TYPE RANGE OF RSDSSELOPT-LOW,  
      LS_RANGE LIKE LINE OF LT_RANGE.  
LOOP AT IT_TABLE ASSIGNING <FIELD_SYMBOL> WHERE FIELDNAME IS NOT INITIAL .  
  LS_RANGE-SIGN      = WMEGC_SIGN_INCLUSIVE.  
  LS_RANGE-OPTION    = WMEGC_OPTION_EQ  
  LS_RANGE-LOW       = <FIELD_SYMBOL>-FIELDNAME.  
  APPEND LS_RANGE TO LT_RANGE.  
ENDLOOP.
```

### **\*New Syntax:**

```
DATA(LT_RANGE) = VALUE RSELECTION( FOR <FIELD_SYMBOL> IN IT_TABLE WHERE ( FIELDNAME IS  
NOT INITIAL )  
                                ( SIGN    = WMEGC_SIGN_INCLUSIVE  
                                  OPTION = WMEGC_OPTION_EQ  
                                  LOW     = <FIELD_SYMBOL>-FIELDNAME ) ).
```



# NESTED FOR WITH MULTIPLE TABLES:

As of ABAP 740, there is a new iteration expression available – FOR. This can be used along with VALUE to populate the desired data. The FOR can be used with Constructor Operators – like VALUE. It is like the LOOP but using the different pattern.

**Example is as below:**

```
* Nested FOR - 3 levels
TYPES:
  BEGIN OF ty_routes_max,
    route   TYPE char10,
    NAME     TYPE char40,
    c_value TYPE char40,
  END OF ty_routes_max.
TYPES: tt_routes_max TYPE STANDARD TABLE OF ty_routes_max
      WITH DEFAULT KEY.

DATA(t_routes_max) =
  VALUE tt_routes_max(
    FOR ls_cust_2 IN t_customres
    FOR ls_route_2 IN t_routes WHERE ( route = ls_cust_2-route )
    FOR ls_rc_2 IN t_rc WHERE ( route = ls_route_2-route
                              AND c_type = 'MAX' )
    ( route = ls_route_2-route
      NAME = ls_route_2-NAME
      c_value = ls_rc_2-c_value )
  ).
```

# USE OF CONVERSION EXITS:

DATA: LV\_MATNR TYPE /SCWM/DE\_MATNR VALUE '0000000000340000001'.

## \* Old Syntax

```
CALL FUNCTION 'CONVERSION_EXIT_ALPHA_OUTPUT'  
  EXPORTING  
    INPUT  = LV_MATNR  
  IMPORTING  
    OUTPUT = LV_MAT_OUT.
```

## \* New Syntax

```
DATA(LV_MAT_OUT) = |{ LV_MATNR ALPHA = OUT }|.
```

# USE OF CONVERSION EXITS: (CONT...)

## \* Old Syntax

CALL FUNCTION 'CONVERSION\_EXIT\_ALPHA\_INPUT'

EXPORTING

INPUT = LV\_MAT\_OUT

IMPORTING

OUTPUT = LV\_MAT\_IN

## \* New Syntax

DATA(LV\_MAT\_IN) = |{ LV\_MAT\_OUT ALPHA = IN }|.

# DECLARATION OF CLASS AND INSTANCE OF OBJECT:

## \* Old Syntax

```
DATA: LR_BW_INB_LOG TYPE REF TO ZCL_EWM_INB_HU_LOGGING.  
CREATE OBJECT LR_BW_INB_LOG.
```

## \* New Syntax

```
DATA(LR_BW_INB_LOG) = NEW ZCL_EWM_INB_HU_LOGGING( ).
```

# USE OF CONCATENATION STATEMENT:

## \* Old Syntax

```
DATA LV_OUTPUT TYPE STRING.
```

```
CONCATENATE 'HELLO' 'WORLD' INTO LV_OUTPUT SEPARATED BY  
SPACE.
```

## \* New Syntax

```
DATA(LV_OUT) = |HELLO| & | | & |WORLD|.
```

# NEW COMMAND FOR TRANSLATE

```
DATA: LV_TEXT(20) TYPE C VALUE 'ACCENTURE'.
```

## **\*Old Syntax:**

```
TRANSLATE LV_TEXT INTO LOWER CASE.
```

```
TRANSLATE LV_TEXT INTO UPPER CASE.
```

## **\*New Syntax:**

```
DATA(LV_LOWER) = |{ LV_TEXT CASE = LOWER }|.
```

```
DATA(LV_UPPER) = |{ LV_TEXT CASE = UPPER }|.
```

# NEW SYNTAX FOR CHECKING SINGLE RECORD IN DB

Initially there was no way of checking if a record is present in DB or not but instead we use SELECT SINGLE \* or up to 1 rows in SQL query. This statement transfers a single record or a single column value from DB to the application server memory. Now we have a new SQL syntax that checks if a record is present in the DB or not without data transfer.

```
SELECT SINGLE @ABAP_TRUE FROM /SCWM/HUHDR INTO @DATA(LV_FLAG) WHERE
                                         GUID_HU = '005056A87B28IED6B8F7B4F9D373E506'.

IF LV_FLAG EQ ABAP_TRUE.
  WRITE:/ 'RECORD EXISTS IN DB'.
ELSE.
  WRITE:/ 'RECORD NOT FOUND'.
ENDIF.
```

# SWITCH – CONDITIONAL OPERATOR

PARAMETERS: P\_VALUE TYPE C.

**\*Old Syntax:**

CASE P\_VALUE.

WHEN 'A'

LV\_NUM = 1

WHEN 'B'

LV\_NUM = 2

WHEN 'C'

LV\_NUM = 3

WHEN 'D'

LV\_NUM = 4

ENDCASE.

**\*New Syntax:**

DATA(LV\_NUM) = SWITCH #( P\_VALUE WHEN 'A' THEN 1  
WHEN 'B' THEN 2  
WHEN 'C' THEN 3  
WHEN 'D' THEN 4 ).

WRITE: LV\_NUM.



# USE OF LINE\_INDEX( )-

- **LINE\_INDEX( )** – Gets the index of the record where the first key value found.

```
" LINE_INDEX( ) - returns the index number of the entry in the internal table  
lv_indx = line_index( lt[ carrid = 'LH' ] ).
```

**LINE\_INDEX( )** – If record with specified key not found then **LINE\_INDEX( )** fails.

# NEW WAY OF USING CORRESPONDING IN ABAP

The new use of CORRESPONDING key word with MAPPING & EXCEPT combination allows more powerful data transfer between two structures. The below post shows some variants of it:

## **\*Old Syntax:**

```
MOVE-CORRESPONDING LS_ADD1 TO LS_ADD2.
```

## **\*New Syntax:**

```
DATA(LS_ADD2) = CORRESPONDING ADD2( LS_ADD1 ).
```

**Note:** Only matching fields are passed from LS\_ADD1 to LS\_ADD2 structure.

# MOVE-CORRESPONDING FOR INTERNAL TABLES

- ABAP Release 7.40 provides many different powerful features. Below examples describes about how to use MOVE-CORRESPONDING with respect to Internal tables:

- MOVE-CORRESPONDING LT\_SCARR TO LT\_SPFLI .

**Note:** This statement just deletes the entries from the target table and moves the matching fields from source to the target .

- MOVE-CORRESPONDING LT\_SCARR TO LT\_SPFLI KEEPING TARGET LINES.

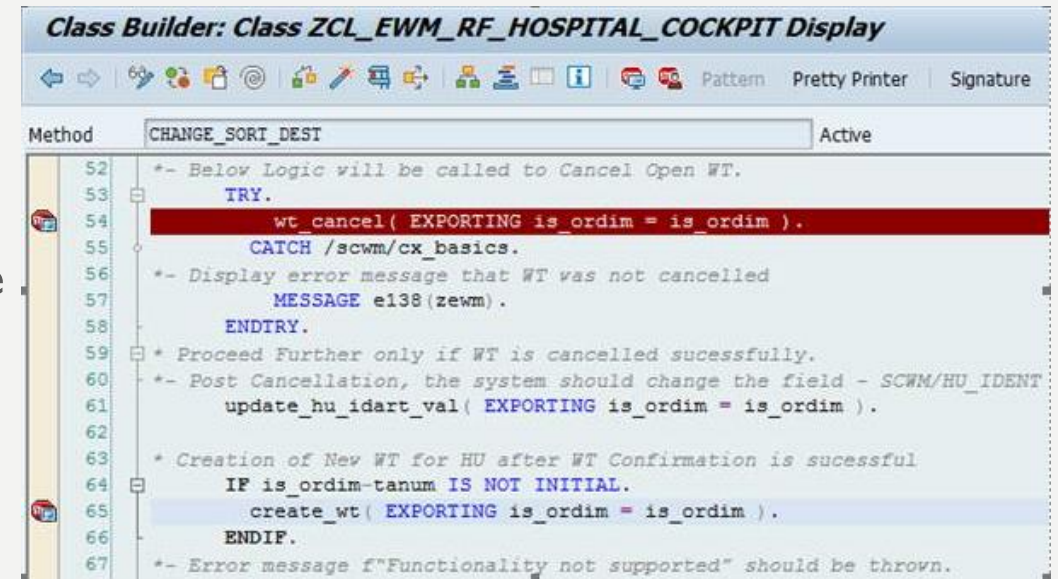
**Note:** This statement just append records from the source table to the target table with matching fields without deleting the target table entries.

# MEASURE & ANALYZE PERFORMANCE OF A SPECIFIC BLOCK OF CODE

- While there are multiple ways to measure performance of a set of lines in a program, the new debugger offers a very easy way of doing this.

## Steps:

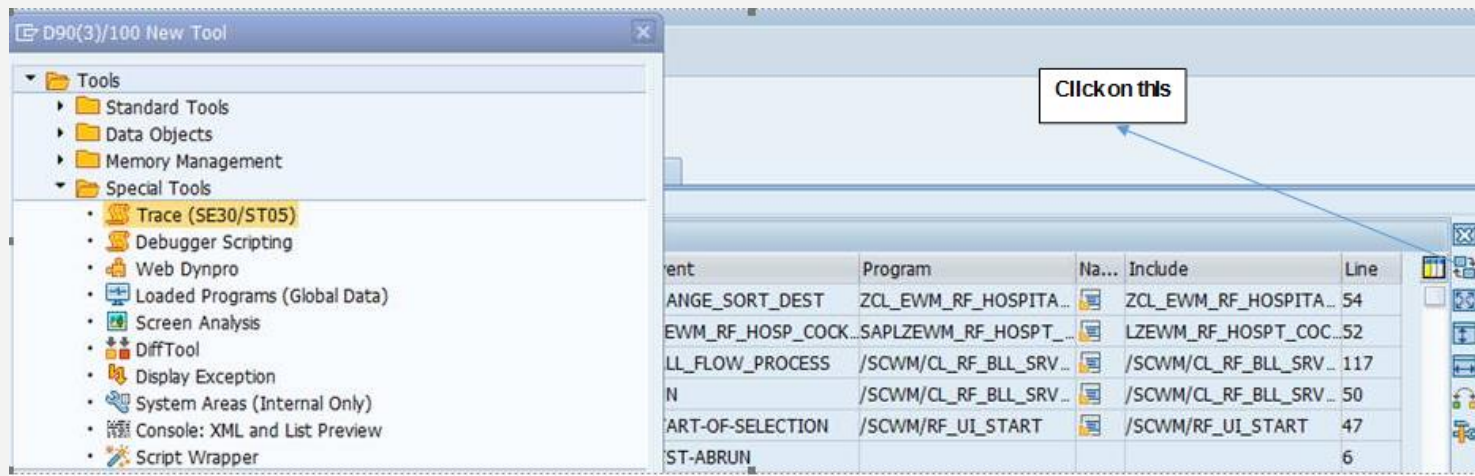
- Set a breakpoint in your class / report / ..., which has the set of lines that you want to analyze.
- Once the debugger session is active, set two breakpoints demarcating these lines. In this case, I want to analyze what is going-on between 54 and 65.
- In the tools pane on the right side, click on New Tool -> Special Tools -> Trace



Class Builder: Class ZCL\_EWM\_RF\_HOSPITAL\_COCKPIT Display

Method: CHANGE\_SORT\_DEST Active

```
52  *~ Below Logic will be called to Cancel Open WT.
53  TRY.
54  wt_cancel( EXPORTING is_ordim = is_ordim ).
55  CATCH /scwm/cx_basics.
56  *~ Display error message that WT was not cancelled
57  MESSAGE e138(zewm).
58  ENDTRY.
59  * Proceed Further only if WT is cancelled successfully.
60  *~ Post Cancellation, the system should change the field - SCWM/HU_IDENT
61  update_hu_idart_val( EXPORTING is_ordim = is_ordim ).
62
63  * Creation of New WT for HU after WT Confirmation is successful
64  IF is_ordim-tanum IS NOT INITIAL.
65  create_wt( EXPORTING is_ordim = is_ordim ).
66  ENDIF.
67  *~ Error message f"Functionality not supported" should be thrown.
```



# **MEASURE & ANALYZE PERFORMANCE OF A SPECIFIC BLOCK OF CODE**

- Voila! You can activate any of the below trace tools from here. Execute the program till the first debug point and then activate the trace. Here, I have activated SQL Trace (ST05)  
You could also use SQL / ABAP / Performance as your situation demands.

The screenshot displays the SAP ABAP IDE interface. The main editor shows the following code:

```
49      MESSAGE e136(zewm_rf) .
50      ENDIF.
51
52      *- Below Logic will be called to Cancel Open WT.
53      TRY.
54          wt_cancel( EXPORTING is_ordim = is_ordim ).
55      CATCH /scwm/cx_basics.
56      *- Display error message that WT was not cancelled
57          MESSAGE e138(zewm) .
58      ENDTRY.
59      *- Proceed Further only if WT is cancelled successfully.
60      *- Post Cancellation, the system should change the field - SCWM/HU_IDENT value i
61          update_hu_idart_val( EXPORTING is_ordim = is_ordim ).
62
63      *- Creation of New WT for HU after WT Confirmation is successful
64      IF is_ordim-tanum IS NOT INITIAL.
65          create_wt( EXPORTING is_ordim = is_ordim ).
66      ENDIF.
```

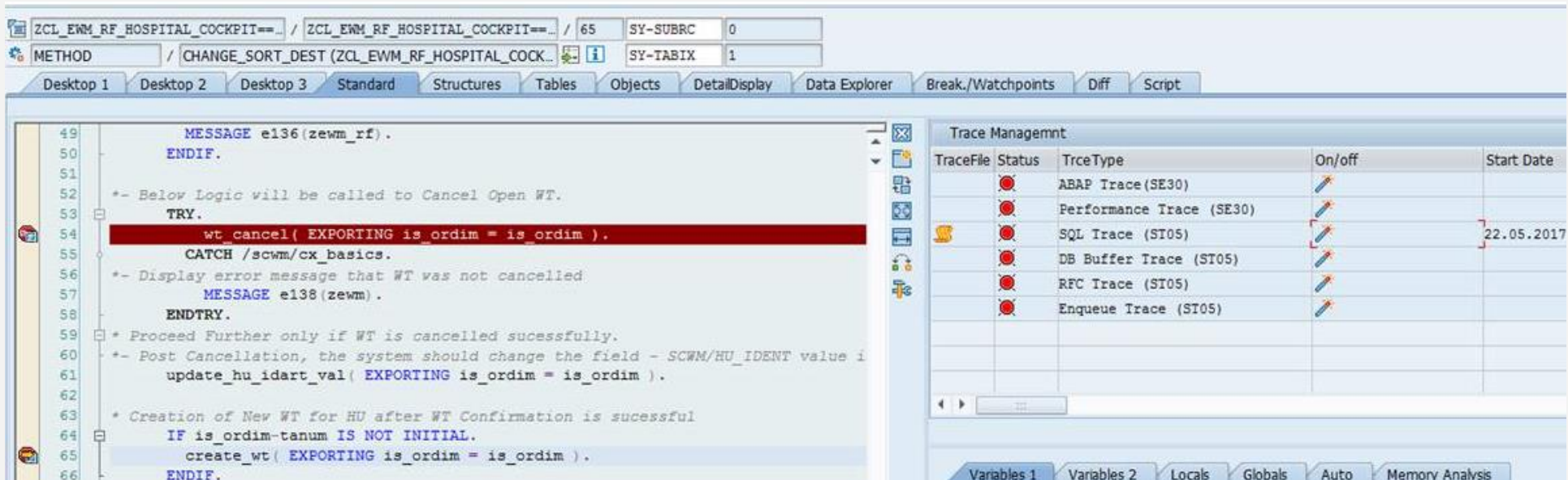
The right-hand pane shows the 'Trace Management' window with the following table:

| TraceFile | Status | TraceType                | On/off | Start |
|-----------|--------|--------------------------|--------|-------|
|           |        | ABAP Trace (SE30)        |        |       |
|           |        | Performance Trace (SE30) |        |       |
|           |        | SQL Trace (ST05)         |        | 22.05 |
|           |        | DB Buffer Trace (ST05)   |        |       |
|           |        | RFC Trace (ST05)         |        |       |
|           |        | Enqueue Trace (ST05)     |        |       |



# ***MEASURE & ANALYZE PERFORMANCE OF A SPECIFIC BLOCK OF CODE***

- Run the program till the next breakpoint and then deactivate the trace by clicking on the 'On/off' button.
- Now the trace file becomes available which can be analyzed further.
- The advantage of this approach is that it supports coding for performance whereby you can develop, analyze it in small parts and fine tune it.



# **WAIT UP TO...SECS COMMAND VS FM: ENQUE\_SLEEP**

- **ABAP Command – WAIT UP TO... Secs** holds the program processing for that seconds but it internally triggers a data base commit which especially can't be used when we have already opened some DB cursor.
- In such a situation **ABAP** provides some **SLEEP** function like **JAVA** by the Function Module: **ENQUE\_SLEEP** which holds the program processing for some moment without any DB commit.
- Instead of **WAIT** command use **ENQUE\_SLEEP** function module which only breaks the program processing for that second but no commit work.



**THANK YOU!**

Questions?

